Лекция по основам Kotlin

1. Переменные

Переменная — это именованная область памяти для хранения данных, значение которой может изменяться во время выполнения программы.

Синтаксис и реализация в Kotlin:

В Kotlin есть два ключевых слова для объявления переменных:

– **val (value)** — для неизменяемых переменных (только для чтения). Присвоить значение можно лишь один раз.

– **var (variable)** — для изменяемых переменных (для чтения и записи).

Тип переменной указывается после имени через двоеточие. Часто тип можно не указывать явно, если он может быть выведен из значения (это называется выводом типов).

Объявление переменных

```
// Объявление с явным указанием типа
val name: String = "Анна"
// Объявление с выводом типа (компилятор сам поймет, что это String)
var age = 25

// name = "Мария" // Ошибка компиляции
age = 26 // OK: var можно перезаписывать
```

Используйте val по умолчанию, чтобы избежать случайного изменения состояния и писать более безопасный код. К var прибегайте только когда изменение значения действительно необходимо.

2. Основные типы данных

Kotlin — статически типизированный язык. Все переменные имеют тип, известный на этапе компиляции.

Целые числа

Определение: Типы для хранения целых чисел без дробной части. Различаются по размеру памяти и, как следствие, диапазону хранимых значений.

Тип	Размер (бит)	Диапазон
Byte	8	от -128 до 127
Short	16	от -32 768 до 32 767
Int	32	от -2^31 до 2^31-1 (около -2.14 млрд до 2.14 млрд)
Long	64	от -2^63 до 2^63-1 (около -9.22 * 10^18 до 9.22 * 10^18)

Числа с плавающей точкой

Типы для хранения чисел с дробной частью.

Числа с плавающей точкой

```
val pi: Double = 3.14159 // Тип по умолчанию, суффикс не нужен
val e: Float = 2.71828f // Суффикс 'f' или 'F' обязателен для Float
val tenDouble = 10.0 // Выведется как Double
```

Булевый тип (Boolean)

Логический тип, может принимать только два значения: true (истина) или false (ложь).

Булевый тип

```
val isReady: Boolean = true
val isEmpty: Boolean = false
```

Булевы значения часто являются результатом операций сравнения

Булевые значения

```
val isGreater = (5 > 3) // true
```

Символы (Char)

Тип для хранения одиночного символа. Представляет собой 16-битное беззнаковое целое число (Unicode code point).

Символы

```
val firstLetter: Char = 'A'
val digit: Char = '9'
val unicodeChar: Char = '\u0041' // Символ 'A' в Unicode
// val error: Char = 'AB' // Ошибка: можно использовать только один символ
```

В Kotlin Char не является числом и не может быть использован напрямую в арифметических операциях (в отличие от Java).

Строки (String)

Строка — это последовательность символов. В Kotlin строки неизменяемы.

Строки объявляются в двойных кавычках " ".

Строка

```
val greeting: String = "Привет, мир!"
```

Строковые шаблоны: Одна из мощнейших возможностей Kotlin. Позволяют вставлять значения переменных и выражений прямо в строку с помощью символа $.

Строковые шаблоны

```
val name = "Иван"
println("Привет, $name!") // Привет, Иван!
val length = name.length
println("Длина твоего имени: ${name.length} символа.") // Для выражений используем {}
// Выведет: Длина твоего имени: 4 символа.
```

Многострочные строки: обрамляются тройными кавычками """ """. Сохраняют все символы форматирования.

Многострочные строки

```
val multiLine = """
    Это многострочная строка.
    Она сохраняет все отступы и
    переносы строк.
        И даже такие большие отступы.
""".trimIndent() // .trimIndent() убирает общие начальные отступы
```

3. null и nullable типы

Null — это фундаментальная особенность Kotlin, призванная устранить проблему NullPointerException (NPE). По умолчанию ни одна переменная не может содержать значение null. Чтобы разрешить null, нужно явно объявить тип как nullable, добавив знак вопроса ? в конце.

Синтаксис:

```
var nonNullableString: String = "Hello"
// nonNullableString = null // ОШИБКА КОМПИЛЯЦИИ: Тип не nullable

var nullableString: String? = "Hello" // Теперь можно присвоить null
nullableString = null // OK
```

Безопасная работа с nullable типами:

Проверка на null вручную (как в Java):

Проверка на null вручную (как в Java):

```
if (nullableString != null) {
    println(nullableString.length) // Компилятор умный, он знает, что здесь не-null
}
```

4. Перевод типов данных

Процесс преобразования значения одного типа в значение другого типа.

Важное отличие от Java: В Kotlin нет неявного преобразования числовых типов. Нельзя просто присвоить Int переменной типа Long, даже если Long "больше".

Преобразование

```
val intValue: Int = 100
// val longValue: Long = intValue // Ошибка компиляции: Type mismatch
```

Для преобразования используются явные функции-члены .toType().

.toType()

```
val intValue: Int = 100
val longValue: Long = intValue.toLong() // Явное преобразование
val doubleValue: Double = intValue.toDouble()

val stringNumber = "123"
val number: Int = stringNumber.toInt() // Парсинг строки в число

// Безопасный парсинг (вернет null, если строка не число)
val safeNumber: Int? = "123abc".toIntOrNull()
```

Ввод и вывод в консоли

Взаимодействие программы с пользователем через консоль. Ввод — получение данных от пользователя, вывод — отображение данных пользователю.

Вывод данных

Для вывода данных в Kotlin используются две основные функции:

print() — выводит переданный аргумент без перевода строки.

println() — выводит переданный аргумент с переводом строки.

Вывод данных — Kotlin

```kotlin
print("Привет, ") // Курсор останется на этой же строке
println("мир!")   // Курсор перейдет на новую строку после вывода

// Вывод:
// Привет, мир!

val name = "Анна"
val age = 25

// Конкатенация строк (менее предпочтительный способ)
println("Имя: " + name + ", Возраст: " + age)

// Строковые шаблоны (String Templates) - рекомендованный способ
println("Имя: $name, Возраст: $age")
println("Через год $name будет ${age + 1} лет") // Выражение в {}

// Форматированный вывод (аналог printf из Java/C)
val pi = 3.14159
println("Значение PI: %.2f".format(pi)) // Округлит до 2 знаков: "3.14"
println("Число: %05d".format(42))       // Дополнит нулями: "00042"
```

Ввод данных

Для чтения данных из консоли используется функция readln() и другие варианты.

readln() — читает всю строку до символа перевода строки и возвращает ее как String. Выбрасывает исключение если поток ввода закрыт (например, при завершении программы).

Ввод данных — Kotlin

```kotlin
print("Введите ваше имя: ")
val name = readln() // Ждет ввода пользователя
println("Привет, $name!")

// Для преобразования в число нужно использовать toInt()/toDouble()
print("Введите ваш возраст: ")
val age = readln().toInt()
println("Через год вам будет ${age + 1}")
```

readlnOrNull() — более безопасная версия. Возвращает String? (nullable строку) или null, если поток ввода закрыт. Рекомендуется для использования.

readlnOrNull() Kotlin

```kotlin
print("Введите число: ")
val input = readlnOrNull() // Тип String?

// Безопасная обработка с помощью Элвис-оператора
val number = input?.toIntOrNull() ?: 0
println("Вы ввели: $number")

// Или с проверкой
if (input != null) {
    val parsedNumber = input.toIntOrNull()
    if (parsedNumber != null) {
        println("Квадрат числа: ${parsedNumber * parsedNumber}")
    } else {
        println("Это не число!")
    }
}
```

readLine() — устаревшая функция из Java, аналогична readlnOrNull(). Возвращает String?.

val input = readLine() // String?

Все, что вводится с клавиатуры, изначально является строкой. Для работы с числами необходимо выполнять преобразование с помощью .toInt(), .toDouble() и т.д., желательно с безопасными вариантами .toIntOrNull().

5. Условные операторы

if

Оператор ветвления, выполняет блок кода, если условие истинно.

Синтаксис:

if

```kotlin
val number = 10
if (number > 0) {
    println("Число положительное")
}
```

if как выражение: В Kotlin if может возвращать значение. Это заменяет тернарный оператор ? : из других языков.

тернарный оператор

```kotlin
val result = if (number > 0) "положительное" else "отрицательное"
println(result)
```

if...else

Расширяет if, добавляя блок кода, который выполняется, если условие ложно.

if...else

```
val score = 85
if (score >= 90) {
    println("Отлично!")
} else if (score >= 75) {
    println("Хорошо")
} else if (score >= 60) {
    println("Удовлетворительно")
} else {
    println("Неудовлетворительно")
}
```

when

Мощный аналог switch из Java и C, но гораздо более выразительный. Проверяет значение на соответствие одному из множества вариантов.

when Kotlin

```
val grade = 'B'
// 1. Простая проверка на совпадение
when (grade) {
    'A' -> println("Отлично!")
    'B' -> println("Хорошо")
    'C' -> println("Удовлетворительно")
    else -> println("Неизвестная оценка") // Аналог 'default'
}

// 2. Проверка нескольких значений в одном case
when (grade) {
    'A', 'B', 'C' -> println("Сдал")
    else -> println("Не сдал")
}

// 3. Проверка на вхождение в диапазон (in) или коллекцию
val number = 15
when (number) {
    in 1..10 -> println("Число от 1 до 10")
    in listOf(11, 12, 13) -> println("Число 11, 12 или 13")
    !in 20..30 -> println("Число не в диапазоне 20..30")
    else -> println("Другое число")
}
```

when Kotlin

```kotlin
// 4. when без аргумента (работает как цепочка if-else-if)
when {
    number < 0 -> println("Отрицательное")
    number % 2 == 0 -> println("Четное")
    else -> println("Положительное и нечетное")
}

// 5. when как выражение
val description = when (number) {
    in 1..10 -> "Маленькое число"
    in 11..100 -> "Среднее число"
    else -> "Большое число"
}
```

6. Циклы

Циклы позволяют выполнять блок кода многократно.

while

Выполняет блок кода пока условие истинно. Проверяет условие до выполнения тела цикла.

while Kotlin

```kotlin
var counter = 5
while (counter > 0) {
    println(counter)
    counter--
}
```

do-while

Выполняет блок кода пока условие истинно. Проверяет условие после выполнения тела цикла. Тело выполнится как минимум один раз.

do-while Kotlin

```kotlin
var userInput: String
do {
    print("Введите 'exit' для выхода: ")
    userInput = readlnOrNull() ?: ""
} while (userInput != "exit")
```

for

Цикл для итерации по всем элементам, предоставляемым итератором (диапазоны, коллекции, массивы и т.д.).

For (Итерация по диапазону (range)) Kotlin

```kotlin
// 1. Итерация по диапазону (range)
for (i in 1..5) { // Включительно: 1, 2, 3, 4, 5
    print("$i ")
}
println()

for (i in 1 until 5) { // Полуоткрытый диапазон: 1, 2, 3, 4
    print("$i ")
}
println()

for (i in 5 downTo 1) { // В обратном порядке: 5, 4, 3, 2, 1
    print("$i ")
}
println()

for (i in 1..10 step 2) { // С шагом 2: 1, 3, 5, 7, 9
    print("$i ")
}
println()
```

For (Итерация по коллекциям/массивам) Kotlin

```kotlin
val fruits = listOf("apple", "banana", "orange")
for (fruit in fruits) { // По элементам
    print("$fruit ")
}
println()

for (index in fruits.indices) { // По индексам
    println("$index: ${fruits[index]}")
}

for ((index, value) in fruits.withIndex()) { // По индексу и значению
    println("$index: $value")
}
```

For (Итерация по Map (словарю)) Kotlin

```kotlin
val map = mapOf(1 to "one", 2 to "two", 3 to "three")
for ((key, value) in map) {
    println("$key = $value")
}
```

7. Управление циклами

Ключевые слова для изменения стандартного потока выполнения циклов.

break — досрочно завершает выполнение всего цикла.

continue — пропускает оставшуюся часть текущей итерации и переходит к следующей.

return — возвращает управление из функции (и из всех циклов внутри нее).

break Kotlin

```kotlin
for (i in 1..10) {
    if (i == 5) break
    print("$i ") // Выведет: 1 2 3 4
}
println()
```

continue Kotlin

```kotlin
for (i in 1..10) {
    if (i % 2 == 0) continue
    print("$i ") // Выведет: 1 3 5 7 9
}
```

8. Коллекции

Коллекции — это контейнеры для хранения групп объектов (элементов). В Kotlin есть два типа коллекций:

Неизменяемые — поддерживают только операции чтения (например, List, Set, Map).

Изменяемые — поддерживают операции и чтения, и записи (добавление, удаление). Имеют префикс Mutable (например, MutableList, MutableSet, MutableMap).

Массивы (Arrays)

Массив — это фиксированная по размеру коллекция элементов одного типа.

Функция arrayOf() Kotlin

```kotlin
val numbers: Array<Int> = arrayOf(1, 2, 3, 4, 5)
val fruits = arrayOf("Apple", "Banana")
```

Фабричная функция Array(size) { lambda } Kotlin

```kotlin
val squares = Array(5) { i -> (i * i) } // [0, 1, 4, 9, 16]
```

Специализированные массивы Kotlin

```kotlin
val intArray: IntArray = intArrayOf(1, 2, 3)
val charArray: CharArray = charArrayOf('a', 'b', 'c')
```

Доступ к элементам: По индексу через оператор [] или функцию .get(index).

Доступ к элементам Kotlin

```kotlin
val firstElement = numbers[0] // Чтение
numbers[1] = 100 // Запись (только для var массивов)
val secondElement = numbers.get(2)
```

Списки (Lists)

Список — это упорядоченная коллекция с доступом к элементам по индексу. Поддерживает дубликаты.

Неизменяемый список (только чтение) Kotlin

```kotlin
val immutableList: List<String> = listOf("a", "b", "c", "a")
println(immutableList[0]) // "a"
// immutableList[0] = "d" // Ошибка: нет операции set
```

Изменяемый список (чтение и запись) Kotlin

```kotlin
val mutableList: MutableList<String> = mutableListOf("a", "b", "c")
mutableList[0] = "d" // OK
mutableList.add("e") // Добавление в конец
mutableList.remove("b") // Удаление элемента
mutableList.add(1, "x") // Вставка по индексу
```

9. Функции

Функция — это блок кода, который выполняет определенную задачу и может быть вызван по имени.

Работа с функциями

Синтаксис объявления: Kotlin

```kotlin
// Ключевое слово fun, затем имя, параметры, возвращаемый тип и тело
fun functionName(parameter1: Type, parameter2: Type): ReturnType {
    // Тело функции
    return result
}
```

Функция без возвращаемого значения Kotlin

```kotlin
fun greet(name: String) {
    println("Привет, $name!")
}
```

Функция с возвращаемым значением Kotlin

```kotlin
fun add(a: Int, b: Int): Int {
    return a + b
}
```

Параметры по умолчанию Kotlin

```kotlin
fun createMessage(text: String, prefix: String = "Info:", suffix: String = "!") {
    println("$prefix $text$suffix")
}
// Вызов:
createMessage("Hello") // "Info: Hello!"
createMessage("Hello", "Warning:", "!!!") // "Warning: Hello!!!"
createMessage(text = "Hello", suffix = "?") // Именованные аргументы: "Info: Hello?"
```

Именованные аргументы (порядок не важен) Kotlin

```kotlin
fun connectToDb(host: String, port: Int, user: String, password: String) { ... }
connectToDb(port = 5432, user = "admin", password = "pass", host = "localhost")
```

10. ООП в Kotlin

Классы и объекты

Класс — это шаблон для создания объектов (экземпляров), который инкапсулирует данные (свойства) и поведение (функции-члены, методы).

Классы и объекты Kotlin

```kotlin
// Объявление класса
class ClassName {
    // Свойства
    var propertyName: Type = initialValue

    // Методы
    fun functionName() { ... }
}

// Создание объекта (экземпляра класса)
val objectName = ClassName()
```

Конструкторы: Специальные функции для инициализации нового объекта.
Первичный конструктор: объявляется прямо в заголовке класса.

Первичный конструктор Kotlin

```kotlin
class Person constructor(firstName: String,
lastName: String) { ... }
// Ключевое слово constructor часто можно опустить
class Person(firstName: String, lastName: String)
{
    // Свойства, инициализированные параметрами
    val fullName = "$firstName $lastName"
}
```

Вторичный конструктор: объявляется внутри тела класса с помощью constructor.

Вторичный конструктор — Kotlin

```kotlin
class Person {
    var name: String = ""
    var age: Int = 0

    constructor(name: String) {
        this.name = name
    }

    constructor(name: String, age: Int) {
        this.name = name
        this.age = age
    }
}
```

Блок инициализации init: выполняется при создании экземпляра, может содержать код инициализации.

Блок инициализации init — Kotlin

```kotlin
class Person(val name: String, val age: Int) {
    val isAdult: Boolean

    init { // Блок инициализации
        isAdult = age >= 18
        println("Объект Person для $name создан")
    }
}
```

Наследование

Механизм, позволяющий одному классу (дочернему, производному) наследовать свойства и методы другого класса (родительского, базового).

По умолчанию все классы в Kotlin final (не могут быть унаследованы). Чтобы разрешить наследование, класс нужно пометить ключевым словом open.

Наследование — Kotlin

```kotlin
// Базовый (родительский) класс
open class Animal(val name: String) {
    open fun makeSound() { // open разрешает переопределение
        println("$name издает звук")
    }
}

// Дочерний класс. Используется : после имени
class Dog(name: String, val breed: String) : Animal(name) {
    // Переопределение метода (требует ключевого слова override)
    override fun makeSound() {
        println("$name породы $breed лает: Гав-гав!")
    }
}
```

Модификаторы доступа

Ключевые слова, которые определяют видимость (область доступности) классов, свойств и функций.

Модификатор	Видимость
public	Виден везде (по умолчанию).
internal	Виден внутри модуля.
protected	Виден внутри класса и его наследников.
private	Виден только внутри области объявления (файла для top-level, класса для членов класса).

Модификаторы доступа — Kotlin

```kotlin
open class Example {
    public val a = 1
    internal val b = 2
    protected val c = 3
    private val d = 4

    fun test() {
        println(d) // Доступно внутри класса
    }
}

class SubExample : Example() {
    fun testSub() {
        println(a) // public - доступно
        println(b) // internal - доступно (если в том же модуле)
        println(c) // protected - доступно в наследнике
        // println(d) // private - НЕ доступно
    }
}

fun main() {
    val example = Example()
    println(example.a) // public - доступно
    println(example.b) // internal - доступно (если в том же модуле)
    // println(example.c) // protected - НЕ доступно извне
    // println(example.d) // private - НЕ доступно извне
}
```

Полиморфизм

Возможность объектов с одинаковой спецификацией (одинаковым родителем или интерфейсом) иметь различную реализацию.

Реализация: через переопределение (override) методов базового класса в дочерних классах.

Полиморфизм Kotlin

```kotlin
open class Shape {
    open fun draw() { // Метод, который будет переопределен
        println("Рисую фигуру")
    }
}

class Circle : Shape() {
    override fun draw() { // Своя реализация для Circle
        println("Рисую круг")
    }
}

class Square : Shape() {
    override fun draw() { // Своя реализация для Square
        println("Рисую квадрат")
    }
}

fun main() {
    val shapes: List<Shape> = listOf(Circle(), Square(), Circle())

    for (shape in shapes) {
        shape.draw() // Вызовется своя реализация для каждого объекта
    }
}
// Вывод:
// Рисую круг
// Рисую квадрат
// Рисую круг
```

Интерфейсы

Интерфейс — это контракт, который определяет, что должен делать класс (какие методы реализовать), но не как он это делает. Класс может реализовывать несколько интерфейсов.

Интерфейс Kotlin

```kotlin
// Объявление интерфейса
interface Drawable {
    fun draw() // Абстрактный метод (без реализации)

    // Метод с реализацией по умолчанию (появился в Kotlin)
    fun highlight() {
        println("Объект подсвечен")
    }
}

interface Clickable {
    fun click()
}

// Класс реализует интерфейсы
class Button : Drawable, Clickable {
    override fun draw() { // Обязательная реализация абстрактного метода
        println("Рисую кнопку")
    }

    override fun click() {
        println("Кнопка нажата")
    }
    // highlight() не переопределен, будет использована реализация по умолчанию
}

fun main() {
    val button = Button()
    button.draw()
    button.click()
    button.highlight()
}
```

Абстрактные классы

Классы, которые обозначаются как abstract. Они не могут быть instantiated (создан их экземпляр). Могут содержать как абстрактные члены (без реализации), так и обычные (с реализацией). Отличаются от интерфейсов тем, что хранят состояние (свойства с значениями).

Абстрактные классы — Kotlin

```kotlin
abstract class Vehicle(val name: String) {
    // Абстрактное свойство (не имеет значения, должно быть переопределено)
    abstract val maxSpeed: Double

    // Обычное свойство
    var currentSpeed: Double = 0.0

    // Абстрактный метод (без тела, должен быть переопределен)
    abstract fun start()

    // Обычный метод (с реализацией)
    fun stop() {
        currentSpeed = 0.0
        println("$name остановился")
    }
}

class Car(name: String, override val maxSpeed: Double) : Vehicle(name) {
    override fun start() { // Реализация абстрактного метода
        println("Заводим двигатель автомобиля $name")
        currentSpeed = 10.0
    }
}

fun main() {
    // val vehicle = Vehicle() // Ошибка: нельзя создать экземпляр абстрактного класса

    val car = Car("Toyota", 200.0)
    car.start()
    car.stop()
}
```